SnackBot

Phase 1: what changed and why

Every change from Phase 0 to Phase 1, file by file: the code before and after, what each new line does, why it was needed, and what it lets you see in Phoenix.

Project folder: `D:\Agentic AI\snackbot-observability-lab`

Read after the Phase 0 code walkthrough. To see the raw changes yourself: `git diff phase-0`

Contents

1. The problem Phase 1 solves 3
2. Summary of every change 6
3. requirements.txt, config.py and .env.example 7
4. tracing.py: the new file 8
5. prompts.py: versioning the prompt 11
6. retriever.py: seeing what the search found 12
7. tools.py: a span for every tool call 13
8. agent.py: one trace per question 15
9. llm.py didn't change: how auto-instrumentation works 18
10. The front doors and the batch script 19
11. Following one question through the new code 21
12. Good to know 22

1. The problem Phase 1 solves

At the end of Phase 0 you could see only two things: the question and the final answer. Everything in between was invisible:

- which chunks the search returned, and whether the right document was among them;
- which tools the LLM called, with which arguments, and what came back;
- how many LLM calls happened, how many tokens each used, and which step was slow;
- what actually broke when SnackBot said "Sorry, something went wrong";
- which answers belonged to the same conversation or the same user.

Phase 1 adds **tracing**: the app now records every step of every question, with its inputs, outputs, timing and errors, and sends that record to **Phoenix**, a local web app where you can browse it. The app's *behaviour* doesn't change at all. SnackBot gives exactly the same answers. It just stops being a black box.

1.1 Six words you need

Term	Meaning	In SnackBot
Span	One timed step: a name, start and end time, inputs, outputs, extra data, and a status (OK / ERROR / unset).	One LLM call, one tool call, one search
Trace	All the spans for one request, linked into a tree by parent/child links.	One question
Attributes	Key/value data attached to a span.	tool.name = check_inventory
Span kind	A label saying what type of step a span is (OpenInference adds this).	AGENT, LLM, TOOL, RETRIEVER, EMBEDDING
Session	Several traces that belong to one conversation, linked by session.id.	One chat
Instrumentation	Code that creates spans. *Auto* = a library does it for you. *Manual* = you write it.	Groq calls are auto; everything else is manual

1.2 The tools we added

Package	What it is
OpenTelemetry (OTel)	The industry-standard library and data format for traces. Not AI-specific; used for web services everywhere.
OpenInference	Arize's AI-specific conventions on top of OTel: span kinds (LLM, TOOL...) and standard attribute names (llm.token_count.prompt, retrieval.documents...). Also the *instrumentors* that auto-trace libraries like Groq and OpenAI.
Phoenix	Arize's open-source app that receives OTel spans and shows them as traces. Runs on your computer at localhost:6006.
arize-phoenix-otel	A helper with one function, register(), that wires OTel up to Phoenix.

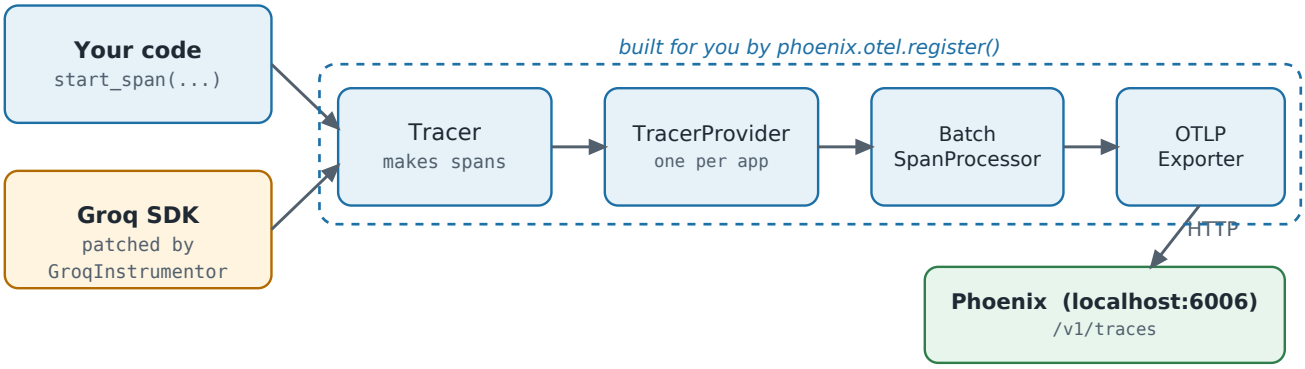


Figure 1. How a span travels from your code to the Phoenix UI.

Read the figure left to right. Your code (and the patched Groq library) asks a **Tracer** to create spans. The **TracerProvider** is the app-wide object that owns the configuration. When a span ends, the **BatchSpanProcessor** puts it in a queue, and a background thread hands batches to the **OTLP exporter**, which sends them over HTTP to Phoenix. OTLP is OpenTelemetry's standard wire format, which is why the same spans could go to Arize AX or any other OTel backend instead.

1.3 What one question looks like now

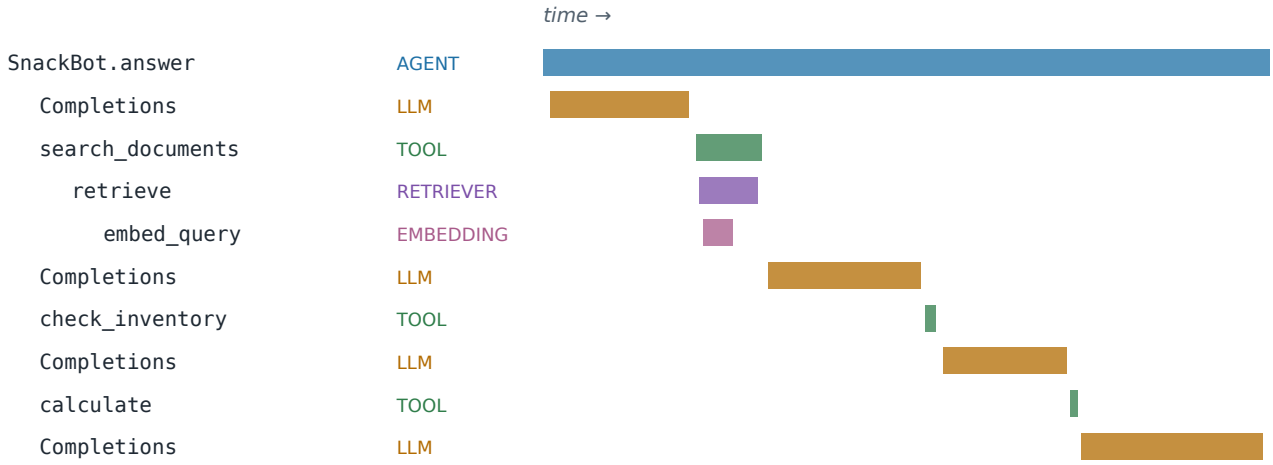


Figure 2. One question as a trace (illustrative timings). Indentation = parent/child; bars = when each span ran.

This is the CW-101 reorder-point question from section 8.2 of the Phase 0 guide. In Phase 0 the four LLM calls and three tool runs happened invisibly. Now each is a span, nested under the AGENT span that covers the whole question. You can click any bar in Phoenix and see exactly what went in and out.

2. Summary of every change

File	Status	In one line
requirements.txt	changed	Adds Phoenix, its OTel helper, the Groq instrumentor and the naming conventions.
snackbot/config.py	changed	Three new settings: TRACING, PHOENIX_ENDPOINT, PHOENIX_PROJECT.
.env.example	changed	Documents the same three settings.
snackbot/tracing.py	**new**	Sets up tracing once; small helpers to create spans, record outputs and errors.
snackbot/prompts.py	changed	Adds PROMPT_VERSION and splits out prompt_variables().
snackbot/retriever.py	changed	RETRIEVER span with every chunk and score; EMBEDDING span for the query.
snackbot/tools.py	changed	TOOL span around every tool call; failed tools turn red.
snackbot/agent.py	changed	AGENT span per question; session/user/metadata/prompt version; errors recorded; returns trace_id and span_id.
snackbot/llm.py	**unchanged**	Groq calls are traced automatically by patching the library.
snackbot/embeddings.py, ingest.py	unchanged	Ingest runs once offline, so it isn't traced.
cli.py, app_streamlit.py, api.py	changed	Start tracing; pass session and user ids; show trace ids.
scripts/run_sample_questions.py	changed	Tags each batch run; flushes spans before exiting.
README.md, learning_notes/01_…	changed / new	Setup steps and the eight labs.

The changes follow one pattern: **wrap each important step in a span, record what goes in and what comes out, and mark the span as an error when something fails**. Every change is marked with a `# PHASE 1` comment in the code.

In the diffs below, `green lines start with +` and were added; `red lines start with -` and were removed; grey lines are unchanged context. Some long unchanged blocks are shortened to `....`

3. requirements.txt, config.py and .env.example

3.1 requirements.txt

requirements.txt

```
 streamlit>=1.38
 fastapi>=0.110
 uvicorn>=0.29
+
+# Phase 1 — tracing
+arize-phoenix>=12.0                       # the Phoenix server + UI (run: phoenix serve)
+arize-phoenix-otel>=0.13                  # register(): one-call OpenTelemetry setup
+openinference-instrumentation-groq>=0.1   # auto-instruments Groq calls as LLM spans
+openinference-semantic-conventions>=0.1   # the standard attribute names (span kinds, documents...)
```

- `arize-phoenix` is the server you start with `phoenix serve`. It's the biggest package. In a company setup the server usually runs centrally (or you use Arize AX), and apps install only the three small packages below it.
- `arize-phoenix-otel` pulls in the OpenTelemetry SDK and exporter for you.
- `openinference-instrumentation-groq` is the piece that makes Groq calls appear as LLM spans without touching llm.py.
- `openinference-semantic-conventions` gives us Python constants such as `SpanAttributes.TOOL_NAME` instead of typing the string `"tool.name"` by hand and risking typos.

3.2 config.py

snackbot/config.py

```
 TOP_K = int(os.getenv("TOP_K", "4"))
+# --- Tracing (Phase 1) ---
+TRACING = os.getenv("TRACING", "full").lower()          # off | auto | full
+PHOENIX_ENDPOINT = os.getenv("PHOENIX_ENDPOINT", "http://localhost:6006")
+PHOENIX_PROJECT = os.getenv("PHOENIX_PROJECT", "snackbot")
+
 # --- Paths ---
```

Setting	What it does	Why it exists
TRACING	off, auto or full (see figure 3).	So you can switch layers on one at a time and see what each adds. In real apps it's also how you turn tracing off in a test environment.
PHOENIX_ENDPOINT	Where the Phoenix server is.	Defaults to your own machine. Would point at a shared server in a team.
PHOENIX_PROJECT	Which project the traces go into in the Phoenix UI.	Keeps experiments apart, e.g. snackbot vs snackbot-test.

`.lower()` makes `TRACING=Full` work the same as `full`. `.env.example` gained the same three lines so you know they exist. Because every setting has a default, your existing `.env` keeps working without changes.

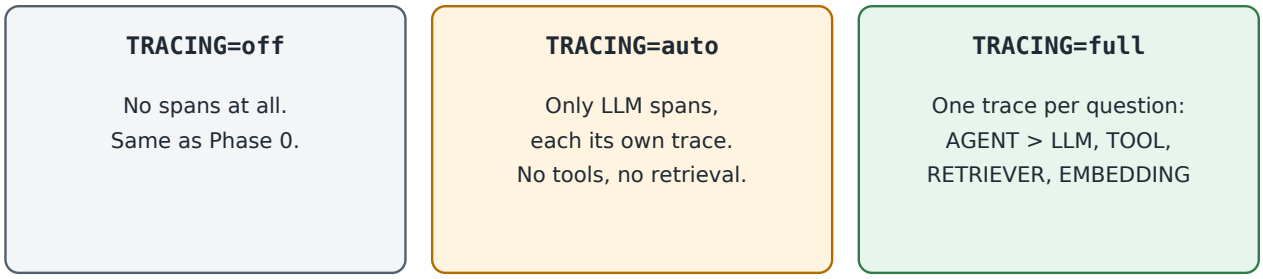


Figure 3. The three tracing levels, chosen with TRACING in .env.

4. tracing.py: the new file

This file holds all the tracing plumbing so the rest of the code only needs a couple of short lines each. It has one setup function and five helpers.

4.1 setup_tracing(): switching tracing on

`snackbot/tracing.py`

```
_provider = None  # the TracerProvider, once set up

def setup_tracing():
    """Call once at program start, before the first LLM call. Safe to call again (no-op)."""
    global _provider
    if _provider is not None or config.TRACING == "off":
        return _provider

    from phoenix.otel import register  # imported here so TRACING=off works without Phoenix

    _provider = register(
        project_name=config.PHOENIX_PROJECT,               # groups traces in the Phoenix UI
        endpoint=f"{config.PHOENIX_ENDPOINT}/v1/traces",   # where spans are sent (OTLP/HTTP)
        auto_instrument=True,  # turns on installed OpenInference instrumentors (here: Groq)
        batch=True,            # send spans in the background in batches
        verbose=False,
    )
    print(f"[tracing] level={config.TRACING} → ...")
    return _provider
```

Line by line:

1. `_provider` is a module-level variable. Python modules are loaded only once per program, so this works as a "has setup already happened?" flag.
2. The `if` makes the function **idempotent**: calling it twice does nothing the second time. That matters for Streamlit, which re-runs the whole script on every click (Phase 0 guide, 9.2). Without this guard, each click would build another pipeline and you'd get duplicate

spans. With `TRACING=off` it returns straight away and nothing is set up.

3. `register(...)` builds the whole pipeline from figure 1 and makes it the **global** TracerProvider, so any code in the program that asks OpenTelemetry for a tracer gets this one.
4. `endpoint=.../v1/traces` is the standard OTLP path on the Phoenix server.
5. `auto_instrument=True` looks for every installed OpenInference instrumentor package and switches it on. We have one, for Groq. If you later `pip install openinference-instrumentation-openai`, OpenAI calls would be traced too, with no code change.
6. `batch=True` uses the BatchSpanProcessor: spans are queued and sent by a background thread every few seconds, so the app never waits on network calls to Phoenix. The alternative, `batch=False`, sends each span immediately and blocks while it does. Simpler, but it slows the app down.

4.2 flush(): don't lose the last spans

```
def flush():
    if _provider is not None:
        _provider.force_flush()
```

Because spans are batched, a few may still be waiting in the queue when a program ends. `force_flush()` sends them right now. The CLI and the batch script call it on exit. Forgetting this is the most common "my last trace is missing" bug. Long-running servers (Streamlit, the API) don't need it, because the background thread keeps sending.

4.3 _tracer(): how auto vs full works

```
def _tracer():
    if config.TRACING != "full":
        return trace.NoOpTracer()
    return trace.get_tracer("snackbot")
```

`trace.get_tracer("snackbot")` returns a real tracer from the global provider that `register()` set up. The name labels where spans came from. `NoOpTracer` is OpenTelemetry's "do nothing" tracer: its spans accept every call but record nothing. So in auto mode, our manual spans silently disappear, while Groq's auto spans (which don't use this function) still appear. That's the whole trick behind the off/auto/full switch, and the rest of the code never has to check the setting.

4.4 start_span(): the helper everything else uses

snackbot/tracing.py

```
@contextmanager
def start_span(name, kind, input_value=None, **attributes):
    with _tracer().start_as_current_span(name) as span:
        span.set_attribute(SpanAttributes.OPENINFERENCE_SPAN_KIND, kind.value)
        if input_value is not None:
            span.set_attribute(SpanAttributes.INPUT_VALUE, input_value)
        for key, value in attributes.items():
            span.set_attribute(key, value)
        yield span
```

Used like `with start_span("retrieve", SpanKind.RETRIEVER, input_value=query) as span:`. Here's what each part does:

- **@contextmanager** turns a function with `yield` into something you can use in a `with` block. Code before `yield` runs when the block starts, and the `with` inside it closes the span when the block ends.
- **start_as_current_span** is the most important call in the whole phase. It creates a span *and makes it the current span*. Any span started inside the block, by our code or by the Groq library, automatically becomes its child. That's how the tree in figure 2 is formed without any code passing parent ids around (see section 9 on context).
- **openinference.span.kind** is the attribute Phoenix reads to pick the icon, colour and special views (a ranked document list for RETRIEVER, a chat view for LLM).
- **input.value** is the standard place for "what went into this step". Phoenix shows it in the span's Input panel.
- ****attributes** lets callers add any extra keys, e.g. `tool.name` or `retrieval.top_k`.
- **Errors for free:** if an exception escapes the block, OTel records it on the span (type, message, stack trace) and sets the status to ERROR, then lets the exception carry on as normal.

4.5 The small helpers

Helper	What it does	Why
set_output(span, value, json_output)	Sets output.value and, for JSON, output.mime_type.	Phoenix shows outputs in an Output panel and pretty-prints JSON.
mark_error(span, exc_or_message)	Records an exception (with stack trace) or a message, and sets status ERROR.	For errors we *catch* and handle: the user gets a polite reply but the trace still shows the failure in red.
ids(span)	Returns the trace id and span id as hex text, or None when tracing is off.	So the app can show or return them: the link between a user's complaint and the exact trace.
to_json(obj)	json.dumps with non-ASCII characters kept readable.	Attributes must be simple values, so dicts are stored as JSON text.

Also note `from openinference.semconv.trace import OpenInferenceSpanKindValues as SpanKind`: it's re-exported from tracing.py so other files write `SpanKind.TOOL` instead of the long name.

5. prompts.py: versioning the prompt

`snackbot/prompts.py`

```
 from datetime import date
+# PHASE 1: bump this whenever you edit the prompt. It's attached to every trace,
+# so you can later filter and compare traces by prompt version.
+PROMPT_VERSION = "v1"
+
 SYSTEM_PROMPT = """You are SnackBot, ...
 ...
 """
+def prompt_variables() -> dict:
+    return {"today": date.today().isoformat()}
+
+
 def system_prompt() -> str:
-    return SYSTEM_PROMPT.format(today=date.today().isoformat())
+    return SYSTEM_PROMPT.format(**prompt_variables())
```

- **PROMPT_VERSION**: prompts change constantly, and a small wording change can quietly make answers worse. Recording the version on every trace means you can filter `llm.prompt_template.version == "v2"` and compare it with v1. You have to bump it by hand when you edit the prompt; Phase 4 covers proper prompt management.
- **prompt_variables()**: OpenInference records a prompt as *template + variables* separately (`{today}` and its value). This was split out so the agent can pass the variables to the tracer without repeating the date logic. `**prompt_variables()` unpacks the dict

into `format(today=...)`, so the result is identical to before.

> **What you see in Phoenix because of it**
>
> On every LLM span: `llm.prompt_template.template`, `llm.prompt_template.variables` and `llm.prompt_template.version`. In Phoenix's prompt view you see the template with the variables filled in.

6. retriever.py: seeing what the search found

snackbot/retriever.py (the logic itself is unchanged; it's just indented inside the spans)

```
 import numpy as np
+from openinference.semconv.trace import DocumentAttributes, EmbeddingAttributes, SpanAttributes
 from snackbot import config
 from snackbot.embeddings import embed
+from snackbot.tracing import SpanKind, start_span, to_json
 ...
 def retrieve(query: str, top_k: int = config.TOP_K) -> list[RetrievedChunk]:
-    chunks, vectors = _load_index()
-    query_vec = embed([query])[0]
+    with start_span("retrieve", SpanKind.RETRIEVER, input_value=query,
+                    **{"retrieval.top_k": top_k}) as span:
+        chunks, vectors = _load_index()
+
+        with start_span("embed_query", SpanKind.EMBEDDING, **{
+            SpanAttributes.EMBEDDING_MODEL_NAME: config.EMBED_MODEL,
+            f"{SpanAttributes.EMBEDDING_EMBEDDINGS}.0.{EmbeddingAttributes.EMBEDDING_TEXT}": query,
+        }):
+            query_vec = embed([query])[0]
+
         scores = vectors @ query_vec
         best = np.argsort(-scores)[:top_k]
         results = [RetrievedChunk(...) for i in best]
+
+        for n, r in enumerate(results):
+            prefix = f"{SpanAttributes.RETRIEVAL_DOCUMENTS}.{n}"
+            span.set_attribute(f"{prefix}.{DocumentAttributes.DOCUMENT_ID}", r.chunk_id)
+            span.set_attribute(f"{prefix}.{DocumentAttributes.DOCUMENT_CONTENT}", r.text)
+            span.set_attribute(f"{prefix}.{DocumentAttributes.DOCUMENT_SCORE}", r.score)
+            span.set_attribute(f"{prefix}.{DocumentAttributes.DOCUMENT_METADATA}",
+                               to_json({"doc_id": r.doc_id, "section": r.section}))
         return results
```

6.1 The RETRIEVER span

The whole search now runs inside a span named `retrieve`, with the query as its input and `retrieval.top_k` as an extra attribute. `retrieval.top_k` isn't an official convention; it's our own attribute. You can add any attributes you like. They show up in the Attributes tab and are filterable.

6.2 The documents: why the funny attribute names

OpenTelemetry attributes can only hold simple values (text, numbers, true/false, or lists of those). They can't hold a list of objects like "4 documents, each with an id, text and score". So OpenInference **flattens** it with numbered keys:

```
retrieval.documents.0.document.id        = "07_hr_leave_policy_2026#1"
retrieval.documents.0.document.content   = "HR-POL-03: ... — Paid time off (PTO)\nPTO covers..."
retrieval.documents.0.document.score     = 0.71
retrieval.documents.0.document.metadata  = '{"doc_id": "07_hr_leave_policy_2026", "section": "Paid time..."}'
retrieval.documents.1.document.id        = "08_hr_leave_policy_2021_ARCHIVED#1"
...
```

Phoenix recognises this pattern and rebuilds it into a ranked list in the span's **Documents** tab. The loop builds these names from constants: `SpanAttributes.RETRIEVAL_DOCUMENTS` is `"retrieval.documents"` and `DocumentAttributes.DOCUMENT_ID` is `"document.id"`. Metadata must be text, so the dict goes through `to_json`.

> **Why this change**
>
> Most bad RAG answers start with bad retrieval. Before this, you couldn't tell whether the LLM *never saw* the right paragraph or *saw it and ignored it*. Those two problems have completely different fixes (better chunking or search vs a better prompt or model). The scores also tell you how confident the search was. In Phase 2, retrieval evaluations grade exactly these documents for relevance.

6.3 The EMBEDDING span

A child span, `embed_query`, wraps just the embedding call. It records the model name and the text that was embedded, using the same flattened style (`embedding.embeddings.0.embedding.text`). We don't record the 384 numbers themselves; they're large and not useful to read.

> **What you see in Phoenix because of it**
>
> Under every search_documents tool call: a retrieve span with a Documents tab listing the 4 chunks, ranked, with scores and source. Beneath it, embed_query, which shows how much of the search time is spent embedding. The first question of a run is slow here because the model loads, which is now visible instead of a mystery.

7. tools.py: a span for every tool call

`snackbot/tools.py (the three tools and their schemas are unchanged)`

```
+from openinference.semconv.trace import SpanAttributes
+
 from snackbot import config
 from snackbot.retriever import retrieve
+from snackbot.tracing import SpanKind, mark_error, set_output, start_span
```

```
 ...
+_DESCRIPTIONS = {s["function"]["name"]: s["function"]["description"] for s in TOOL_SCHEMAS}
+
+
+def _looks_like_error(result: str) -> str | None:
+    if result.startswith("Error"):
+        return result
+    if result.startswith("{") and '"error"' in result:
+        return json.loads(result).get("error")
+    return None
+
+
 def run_tool(name: str, arguments_json: str) -> str:
-    if name not in TOOL_FUNCTIONS:
-        return f"Error: unknown tool '{name}'."
-    try:
-        args = json.loads(arguments_json or "{}")
-        return TOOL_FUNCTIONS[name](**args)
-    except Exception as exc:
-        return f"Error running {name}: {exc}"
+    with start_span(name, SpanKind.TOOL, input_value=arguments_json or "{}", **{
+        SpanAttributes.INPUT_MIME_TYPE: "application/json",
+        SpanAttributes.TOOL_NAME: name,
+        SpanAttributes.TOOL_DESCRIPTION: _DESCRIPTIONS.get(name, ""),
+        SpanAttributes.TOOL_PARAMETERS: arguments_json or "{}",
+    }) as span:
+        if name not in TOOL_FUNCTIONS:
+            result = f"Error: unknown tool '{name}'."
+        else:
+            try:
+                args = json.loads(arguments_json or "{}")
+                result = TOOL_FUNCTIONS[name](**args)
+            except Exception as exc:
+                mark_error(span, exc)
+                result = f"Error running {name}: {exc}"
+
+        set_output(span, result, json_output=(name == "check_inventory"))
+        problem = _looks_like_error(result)
+        if problem:
+            mark_error(span, problem)
+        return result
```

7.1 Why the span goes in run_tool, not in each tool

Every tool call from the LLM passes through `run_tool`, so one span here covers all three tools, and any tool added later, automatically. The span is named after the tool (`check_inventory`, `calculate`…) so the tree reads naturally.

7.2 What gets recorded

Attribute	Value	Why
input.value + input.mime_type	The raw JSON arguments the LLM produced	Did the LLM pass sensible arguments? (e.g. "Houston", or a misspelled SKU)
tool.name, tool.description	Which tool, and the description the LLM was given	Lets you check whether a vague description caused a wrong tool choice
tool.parameters	The same arguments, in the standard tool slot	What OpenInference-aware views and evals look for
output.value	The exact text sent back to the LLM	Was the tool's data right? Was the search result useful?
output.mime_type	application/json for check_inventory	Phoenix pretty-prints the JSON

`_DESCRIPTIONS` is a small lookup built once from `TOOL_SCHEMAS` so each span can carry the tool's description without repeating it.

7.3 Two kinds of tool failure, both now visible

In Phase 0, tool problems were turned into text for the LLM and nobody else saw them. That design is still right (it lets the LLM recover), but now the span is marked too:

1. **A crash inside a tool** (e.g. the LLM sends a parameter name that doesn't exist, so `**args` fails): caught by the `except`, recorded with `mark_error(span, exc)` including the stack trace, then turned into an error message for the LLM as before.
2. **A tool that "succeeds" but reports a problem**, e.g. `check_inventory` returning `{"error": "Unknown warehouse ..."}` or `calculate` returning `"Error: Only numbers..."`. No exception happens, so OTel wouldn't notice. `_looks_like_error` spots these by their shape and the span is marked ERROR with the message.

> **What you see in Phoenix because of it**
>
> A red check_inventory span with the status message "Unknown warehouse 'Houston'. Options: [...]", even though SnackBot replied politely. The filter status_code == 'ERROR' lists every failed tool call across all questions, which is a quick way to spot a broken tool or a bad tool description.

8. agent.py: one trace per question

This file has the most important changes. Here they are in three parts.

8.1 AgentResult gets two new fields

`snackbot/agent.py`

```
 @dataclass
 class AgentResult:
     answer: str
-    tools_used: list[str] = field(default_factory=list)  # just tool names: deliberately thin
+    tools_used: list[str] = field(default_factory=list)
+    trace_id: str | None = None   # PHASE 1: lets you find this exact run in Phoenix
+    span_id: str | None = None    # PHASE 2 will use this to attach user feedback to the trace
```

The caller now gets the trace id back along with the answer. That's how a bug report ("SnackBot said something wrong at 2:14") turns into a direct link to the exact trace. The span id is for Phase 2: when a user clicks thumbs-down, we'll attach that feedback to this specific span.

8.2 answer() becomes a traced wrapper around _run()

snackbot/agent.py

```
-    def answer(self, question: str, history: list[dict] | None = None) -> AgentResult:
+    def answer(self, question: str, history: list[dict] | None = None, *,
+               session_id: str | None = None, user_id: str | None = None,
+               metadata: dict | None = None, tags: list[str] | None = None) -> AgentResult:
+        with using_attributes(
+            session_id=session_id or "",
+            user_id=user_id or "",
+            metadata={"model": self.model, "prompt_version": PROMPT_VERSION, **(metadata or {})},
+            tags=tags or [],
+            prompt_template=SYSTEM_PROMPT,
+            prompt_template_version=PROMPT_VERSION,
+            prompt_template_variables=prompt_variables(),
+        ), start_span("SnackBot.answer", SpanKind.AGENT, input_value=question, **{
+            SpanAttributes.AGENT_NAME: "SnackBot",
+            "agent.history_turns": len(history or []) // 2,
+        }) as span:
+            result = self._run(question, history, span)
+            result.trace_id, result.span_id = ids(span)
+            set_output(span, result.answer)
+            span.set_attribute("agent.tools_used", result.tools_used)
+            return result
+
+    def _run(self, question, history, span) -> AgentResult:
         messages = [{"role": "system", "content": system_prompt()}]
         ...
```

New parameters

The * in the signature means everything after it must be passed by name (`session_id=...`), which avoids mixing up arguments. They're all optional, so `bot.answer("question")` still works exactly as in Phase 0.

Parameter	Who passes it	What you get in Phoenix
session_id	Each interface: one id per conversation	The Sessions tab groups a chat's traces in order
user_id	Each interface	Filter by user; spot one user hitting many errors

Parameter	Who passes it	What you get in Phoenix
metadata	Each interface, e.g. {"interface": "cli"}	Filter with metadata["interface"] == "api"
tags	The batch script, e.g. the run tag and category	Find all traces from one test run

using_attributes: setting values for every span at once

`using_attributes(...)` doesn't create a span. It puts values into the OpenTelemetry **context** (section 9). Every span created while the block is open, including the Groq LLM spans created inside a library that knows nothing about SnackBot, copies these values onto itself. That's why you can filter LLM spans by `session.id` even though llm.py never mentions sessions. The model and prompt version are always added to the metadata, so every trace records which configuration produced it.

The AGENT span

`start_span("SnackBot.answer", SpanKind.AGENT, ...)` opens the **root span**: the top of the tree. Because it's the current span while the loop runs, every LLM call and tool call becomes its child, and they all share one trace id. Without it, you get what you see in `TRACING=auto` mode: each LLM call as a separate, disconnected trace.

The attributes `agent.name`, `agent.history_turns` (how many earlier turns were sent) and, at the end, `agent.tools_used` and the output, give you a one-line summary of the whole run on the root span.

Why split into answer() and _run()?

The original loop has several `return` statements (final answer, too many steps, error). Moving the loop into `_run()` means `answer()` can do the tracing work (ids, output, tools used) once, after any of those returns, instead of repeating it at each `return`. `_run` receives the span so it can add attributes and mark errors.

8.3 Changes inside the loop

`snackbot/agent.py, inside _run()`

```
         try:
-            for _ in range(config.MAX_AGENT_STEPS):
-                response = chat(messages, tools=TOOL_SCHEMAS, model=self.model)
+            for step in range(1, config.MAX_AGENT_STEPS + 1):
+                span.set_attribute("agent.steps", step)
+                response = chat(messages, tools=TOOL_SCHEMAS, model=self.model)  # auto-traced LLM span
                 ...
-                    result = run_tool(tc.function.name, tc.function.arguments)
+                    result = run_tool(tc.function.name, tc.function.arguments)  # TOOL span
                 ...
+            mark_error(span, f"Hit MAX_AGENT_STEPS={config.MAX_AGENT_STEPS} without a final answer")
             return AgentResult("Sorry, I couldn't finish answering that (too many steps).", tools_used)
-        except Exception:  # PHASE 0: errors are swallowed. Painful on purpose.
+        except Exception as exc:
+            mark_error(span, exc)
```

```
            return AgentResult("Sorry, something went wrong. Please try again.", tools_used)
```

- **agent.steps** is updated each pass, so the root span shows how many LLM calls the question needed. A question that normally takes 2 steps suddenly taking 6 is a strong hint the LLM is confused or looping.
- **Too many steps** is now marked as an error, so these runs show up as failures instead of looking like normal answers.
- **The swallowed exception** is the biggest single fix. The user still sees the same polite message (we chose not to change the user experience), but `mark_error(span, exc)` records the real exception type, message and stack trace on the root span. In testing, a Groq `tool_use_failed` error (the model produced a badly formed tool call) showed up as a red AGENT span with a red LLM span beneath it, with Groq's exact error message.
- Why `mark_error` and not just let the exception propagate? Because we catch it. The automatic error recording in `start_span` only triggers for exceptions that escape the block, so a caught one has to be recorded by hand. This applies to any code that catches errors.

9. llm.py didn't change: how auto-instrumentation works

llm.py still just calls `client().chat.completions.create(**kwargs)`. So where do the LLM spans come from?

When `register(auto_instrument=True)` runs, it activates `GroqInstrumentor`. The instrumentor **monkey-patches** the Groq library: it replaces Groq's `Completions.create` method with a wrapper. The wrapper:

1. starts an LLM span (named Completions, after the Groq class it wraps) as a child of whatever span is current;
2. records the input messages, model name, temperature and the tool schemas;
3. calls the real Groq method;
4. records the reply (text or tool calls), finish reason and token counts from `response.usage`, then ends the span. If Groq raises an error, it records that instead.

This is why setup must happen **before the first LLM call**: the patch has to be in place before Groq is used. It also explains why the retries in llm.py are visible: each retry attempt is a separate call to the patched method, so a rate-limited question shows several LLM spans, the failed ones in red.

Context: the idea that makes it all connect

OpenTelemetry keeps a "current context" for the running code (using Python's contextvars). **start_as_current_span** stores the new span there while its block runs, and puts the previous one back when the block ends. Any new span looks in the context to find its parent. **using_attributes** stores session id, user id and so on in the same context.

So when the Groq wrapper creates its LLM span deep inside llm.py, it finds the AGENT span as its parent and the session id in the context, without any SnackBot code passing them along. This is called **context propagation**. It's also why the RETRIEVER span correctly nests under the search_documents TOOL span: retrieve() runs inside run_tool's span block.

10. The front doors and the batch script

10.1 cli.py

cli.py

```
+import getpass
+import uuid
+
+from snackbot import config
 from snackbot.agent import SnackBot
+from snackbot.tracing import flush, setup_tracing
 def main():
+    setup_tracing()
     bot, history = SnackBot(), []
+    session_id = str(uuid.uuid4())
+    user_id = getpass.getuser()
     ...
         if question == "/reset":
-            history = []
+            history, session_id = [], str(uuid.uuid4())
         ...
-        result = bot.answer(question, history)
+        result = bot.answer(question, history, session_id=session_id, user_id=user_id,
+                            metadata={"interface": "cli"})
         print(f"\nSnackBot: {result.answer}\n")
+        if result.trace_id and config.TRACING == "full":
+            print(f"  (trace {result.trace_id})\n")
         ...
+    flush()
```

- `setup_tracing()` comes first, before any LLM call, for the patching reason in section 9.
- `uuid.uuid4()` makes a random unique id. One per conversation, and `/reset` starts a new one, so a reset conversation shows as a new session in Phoenix.
- `getpass.getuser()` is your Windows username, used as the user id.
- The trace id is printed after each answer so you can search for it in Phoenix.
- `flush()` on exit sends the last batched spans before the program closes (section 4.2).

10.2 app_streamlit.py

Same idea, adapted to Streamlit: `setup_tracing()` at the top of the script (safe on every re-run thanks to the guard in 4.1); the session id is kept in `st.session_state` so it survives re-runs, and "New conversation" makes a new one; a sidebar box lets you type a user id, so you can pretend to be different users; the sidebar shows the tracing level, and each answer shows its trace id. No `flush()` is needed, because the server keeps running.

10.3 api.py

api.py

```
+setup_tracing()
 app = FastAPI(title="SnackBot API")
 ...
 class ChatRequest(BaseModel):
     message: str
     session_id: str | None = None
-    user_id: str | None = None  # unused for now; Phase 1 attaches it to traces
+    user_id: str | None = None
 class ChatResponse(BaseModel):
     answer: str
     session_id: str
+    trace_id: str | None = None
+    span_id: str | None = None
 ...
-    result = bot.answer(req.message, history)
+    result = bot.answer(req.message, history, session_id=session_id, user_id=req.user_id,
+                        metadata={"interface": "api"})
```

The API now passes the session and user ids it already received into the trace, and returns the trace id and span id in every response. This is standard practice in production: the front-end keeps the ids so that feedback and bug reports can point at the exact trace.

10.4 scripts/run_sample_questions.py

scripts/run_sample_questions.py

```
+    setup_tracing()
     ...
+    run_tag = f"batch-{time.strftime('%Y%m%d-%H%M%S')}"
     for q in questions:
-        result = bot.answer(q["question"])
+        result = bot.answer(q["question"], user_id="batch-script", tags=[run_tag, q["category"]],
+                            metadata={"interface": "batch", "question_id": q["id"],
+                                      "category": q["category"]})
-        print(f"[{q['id']}] {seconds:4.1f}s  tools={result.tools_used}")
+        print(f"[{q['id']}] {seconds:4.1f}s  tools={result.tools_used}  trace={result.trace_id}")
     ...
+    flush()
-    path = ROOT / "results" / "phase0_answers.md"
+    path = ROOT / "results" / "phase1_answers.md"
```

Each run gets a unique tag such as batch-20261013-141500, and each question carries its id and category in the metadata. In Phoenix you can then filter one run, or one category such as

`metadata["category"] == "conflicting_docs"`. The output file now lists the trace id next to each answer, so you can go straight from a wrong answer to its trace. This is a manual version of what Phase 2 automates with datasets and experiments.

11. Following one question through the new code

Here is the order in which spans open and close for the CW-101 question. Read it with figure 2.

#	Code that runs	Span event
1	cli.py: setup_tracing()	Pipeline built; Groq patched. (No span yet.)
2	agent.answer(): using_attributes(...)	Session, user, metadata and prompt info put in context.
3	agent.answer(): start_span(AGENT)	**SnackBot.answer opens**: the root; a new trace id is created.
4	_run(): chat(...) → patched Groq create()	**Completions (LLM)** opens and closes as a child; records the messages, tool request and tokens.
5	run_tool('search_documents')	**search_documents (TOOL)** opens.
6	→ retrieve()	**retrieve (RETRIEVER)** opens inside the tool span.
7	→ embed()	**embed_query (EMBEDDING)** opens and closes.
8	retrieve() records 4 documents and returns	retrieve closes, then search_documents closes with its output.
9–12	chat → check_inventory → chat → calculate	LLM, TOOL, LLM, TOOL spans, all children of the root.
13	chat(...) returns plain text	Final LLM span; the loop ends.
14	answer(): ids(), set_output(), tools_used	Root span gets its output and summary; **root closes**.
15	(background)	The batch processor sends all 10 spans to Phoenix within a few seconds.

A span is only sent when it *ends*, and the root ends last. That's why a trace sometimes appears in Phoenix a few seconds after the answer is shown.

11.1 What three kinds of error look like now

What went wrong	User sees	Phase 0 trace	Phase 1 trace
Groq error (bad model name, tool_use_failed, rate limit after retries)	"Sorry, something went wrong"	Nothing	Red AGENT span with the exception; red LLM span with Groq's message
Tool problem (Houston, bad SKU, bad expression)	Usually a sensible reply (the LLM recovers)	Nothing	Red TOOL span with the reason; you can check whether the LLM recovered well
Too many steps	"couldn't finish (too many steps)"	Nothing	Red AGENT span with agent.steps = 6; every step visible

12. Good to know

12.1 Frequently asked questions

Question	Answer
Does tracing slow SnackBot down?	Barely. Creating a span takes microseconds and sending is done in the background in batches. The LLM call (seconds) dwarfs it.
What if Phoenix isn't running?	SnackBot still works. The exporter fails to send and logs a warning in the terminal; the spans are dropped. Tracing should never break the app.
Why are successful spans not marked OK?	OTel's default status is UNSET, which means "no error". Phoenix treats UNSET as success. Only failures need marking.
Why is the LLM span called Completions?	The Groq instrumentor names it after the Groq class it wraps. Its kind (LLM) is what matters.
Why isn't ingest traced?	It runs once, offline, not per question. You could trace it the same way; it just isn't where the debugging questions are.
Can I use decorators instead of with-blocks?	Yes. phoenix.otel's tracer offers @tracer.tool, @tracer.chain and @tracer.agent decorators. We used explicit with-blocks so you can see every attribute being set. Both produce the same spans.

12.2 Traces contain your data

Important at work

Traces store full prompts, documents, tool results and answers. In SnackBot that's fictional data, but at a real company it could include confidential or personal information. That's why production setups decide carefully what to record and who can see the traces.

OpenInference has settings to hide data on auto-instrumented spans, for example the environment variables OPENINFERENCE_HIDE_INPUTS and OPENINFERENCE_HIDE_OUTPUTS. These apply to spans made by instrumentors (our LLM spans); our manual spans would need their own masking. Follow your team's rules on what may be traced.

12.3 Check your understanding

1. Why does setup_tracing() need to run before the first Groq call?
2. In TRACING=auto mode, why does each LLM call become its own trace?
3. How does the Groq LLM span get the session id when llm.py never mentions sessions?
4. A tool returns {"error": "No product found"} without raising. How does its span turn red?
5. Why does the CLI call flush() but Streamlit doesn't?
6. Why does the retriever store documents as retrieval.documents.0.document.content and so on?
7. Where would you look to tell "retrieval brought the archived HR policy" apart from "the LLM chose the archived policy over the current one"?

Answers in brief: (1) the instrumentor patches Groq at setup, so earlier calls would be unpatched; (2) our AGENT span is a no-op, so there's no current parent span and each LLM span becomes a root; (3) using_attributes put it in the context, and every span created inside reads it; (4) _looks_like_error spots the error shape and mark_error sets the status; (5) spans are batched and the CLI exits, while the Streamlit server keeps running; (6) OTel attributes can't hold lists of objects, so OpenInference flattens them with numbered keys that Phoenix knows how to rebuild; (7) the retrieve span's Documents tab (was the archived chunk ranked high?) and then the final LLM span (with both in front of it, which did it use?).

Next step

Work through the eight labs in learning_notes/01_phase1_tracing.md. Lab 7, root-causing your Phase 0 grades, is where this all comes together. Then Phase 2 turns those manual judgements into automatic evaluations attached to these same traces.